

ALMA MATER STUDIORUM · UNIVERSITÀ DI BOLOGNA

---

Second Cycle Degree  
Artificial Intelligence

Notes  
Introduction to  
Complexity and Computability

**Student:**  
Matteo Canghiari

**Academic Year 2025/2026**



# Chapter 1

## Conceptual and mathematical preliminaries

First of all, we introduce the notion of **function**:

what a function really is?

### Definition

Given two sets  $X$  and  $Y$ , such that:

$$X \cdot Y = \{(x, y) | x \in X, y \in Y\}$$

a **relation** is a subset of the cartesian product.

A function from  $X$  to  $Y$  is a relation described as:

$$f \subseteq X \cdot Y.$$

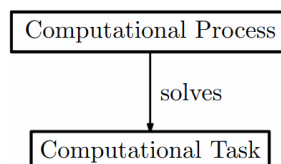
It's a subset of the cartesian product, such that  $\forall x \in X$  there is one  $y \in Y$  such that:

$$(x, y) \in f$$

and we state as follows:  $f : x \rightarrow y$ .

### 1.1 The standard paradigm

Consider the following image:



Processes and tasks are two different entities, that can be informally described as:

- **Computational process**: what we have to do, like a Python program.
- **Computational task**: what we really want to achieve.

There may be several processes for solving the same task, but sometimes the task is so hard to compute that no process can solve it. At the beginning, both of them are dependent from the **context**, but introducing a **model** (*arrow in the representation*) we are able to make them totally independent.

## 1.2 The multiplication problem

### Example

e.g. Let's consider the following task.

**Task**: multiply two natural numbers  $a$  and  $b$ , both expressed by  $n$  digits.

A possible solution could be: reduce the multiplication to a sequence of additions. Therefore, knowing that we have to repeat the addition of number  $a$  for  $b$  times, our process is:

$$a \times b = a + a + \dots + a$$

The amount of time required by this process can be summarized as follows:

$$b \times n$$

since we repeat the addition operation for  $b$  times. We call this process as **RepeatedAdditions**.

Another process that can solve the previous task might be: multiply  $a$  and  $b$  using the basic elementary school algorithm. Let  $a_1, a_2, \dots, a_n$  be the digits of  $a$ , we compute the final result performing  $b \times (a_1 \times 10^0) + \dots + b \times (a_n \times 10^{n-1})$ , which takes a number of steps equal to  $n \times n$ . We call this process as **GridMethod**.

Between these two processes there's a huge difference, that can be described as follows:

- RepeatedAdditions: the term  $b$  can be **exponential** in  $n$ , so  $b$  might be  $10^n$  in the worst case.
- GridMethod: its complexity is quadratic, but anyway we are sure that it takes less time or effort than RepeatedAdditions.

Even though by both processes we can solve the task, RepeatedAdditions approach takes ages. Therefore, they solve the same task in different ways and

not always this is acceptable in terms of efficiency.

### 1.3 What is a process?

In theory of computation, a process is taken to be an **algorithm**, so we replace the term process by the word algorithm. Any algorithm must satisfy always these constraints:

1. It must be a **finite** description of computation steps.
2. Each step of the description must be as **elementary** as possible.
3. The way the next step is achieved must be **deterministic**.

### 1.4 Mathematical preliminaries

#### Definition

The cardinality of any set  $X$  is described by:

$$|X|$$

We will mainly work with discrete numbers:  $N$  the set of natural numbers and  $Z$  the set of integer numbers. Therefore, we avoid  $R$  the set of real numbers, since their cardinalities are infinite.

#### Definition

A condition  $P(n)$  depending on  $n \in N$  holds for  $n$  if there is a  $N \in N$  such that  $P(n)$  holds for every  $n > N$ .

We are defining a kind of *threshold* to determine if a property  $P(n)$  for this  $n$  holds or not.

#### Definition

Given  $S$  a finite set, then a string (*a sequence of characters*) defined over an **alphabet**  $S$  is a finite, ordered, possibly empty, tuple of elements coming from  $S$ .

### Definition

The set of all strings over  $S$  of length exactly  $n \in \mathbb{N}$  is indicated as  $S^n$ , where  $S^0$  is the set containing only the empty string  $\varepsilon$ .

### Definition

The set of all strings over  $S$  is  $\cup_{n=0}^{\infty} S^n$  and is indicated as  $S^*$ .

### Definition

The concatenation of two strings  $x$  and  $y$  over  $S$  (*where  $x$  and  $y$  are defined over the alphabet  $S$* ) is indicated as:

$$xy \text{ or } x \cdot y$$

### Definition

The **length** of a string is indicated as:

$$|x|$$

### Example

e.g Let's consider the following alphabet.

$$\{0, 1\}^2 = \{00, 01, 10, 11\}$$

where:

- $\{0, 1\}^2 \rightarrow$  alphabet
- $\{00, 01, 10, 11\} \rightarrow$  strings defined over the current alphabet

A possible concatenation can be:

$$000101$$

whose cardinality is:

$$|000101| = 6$$

### Useful notation

- For a given real number  $x$ ,  $\lceil x \rceil$  is the smallest element of  $\mathbb{Z}$  such that  $\lceil x \rceil \geq x$ . Whenever a real number  $x$  is used in place of a natural number, we implicitly read it as  $\lceil x \rceil$ .
- For a natural number  $n$ ,  $[n]$  is the set  $\{1, \dots, n\}$ .
- $\log x$  is base 2 logarithm.

## 1.5 Tasks as functions

Summing up, we have stated:

1. A function is not a mapping operation, it is basically a relation between pairs composed by values coming from a domain and a codomain.
2. Any function is usually described by:

$$f : A \rightarrow B$$

where:

- $A \rightarrow$  domain
  - $B \rightarrow$  codomain
3. Sometimes  $A$  and  $B$  are countable not-string sets, therefore we must encode them following some strategies.
  4. The encoding of any element  $x$  is indicated as  $\lfloor x \rfloor$ .

Typically, tasks (*what we want to achieve*) are about: given a function (*relation between two sets*) we turn an input to an output. In some cases, the input and output are not strings, so we need a way to represent them as a sequence of characters, following some encoding approaches.

### Example

e.g. Let's consider the following alphabet.

$$S = \{a, b, c\}$$

any single character of this alphabet can be encoded as:

$$a \rightarrow 00$$

$$b \rightarrow 01$$

$$c \rightarrow 10$$

For instance, the original string *abbc* becomes:

$$00010110$$

Note: we get an injective function but not a surjective function, since 11 from the codomain corresponds to nothing in the domain.

### Definition

Given any natural number  $n$ , we get an encoding equal to transform it in a binary number:

$$12 \rightarrow 1100$$

### Definition

Given pairs of binary strings:

$$\{0, 1\}^* \times \{0, 1\}^*$$

where both  $\{0, 1\}^*$  can be a whatever sequence of binary numbers, we encode the resulting string  $x\#y$  over the alphabet  $\{0, 1, \#\}$ , where  $\#$  acts as **separator**.

In computational theory, a task is interesting if it consists in computing a function from the alphabet  $\{0, 1\}^*$  to itself. An important function that starts from  $\{0, 1\}^*$  and goes to  $\{0, 1\}^*$  are those whose range are strings of length exactly one, named **boolean functions**, also called **characteristic functions**.

### Definition

We identify the characteristic function such a function  $f$  with a subset  $L_f$  of  $\{0, 1\}^*$ , defined as follows:

$$L_f = \{x \in \{0, 1\}^* \mid f(x) = 1\}$$

### Definition

Any subset of  $S^*$  (*the set of any string of any dimension  $n$* ) is called **language**, indicated as  $M$ .

These last definitions are extremely useful, because we can design a **decision problem** for a given language  $M$  (*subset of the alphabet  $\{0, 1\}^*$* ) as task of computing a relation  $f$  such that the language  $M = L_f$ . If we can derive such equivalence we get a resolution of the problem.



# Chapter 2

## Computational model

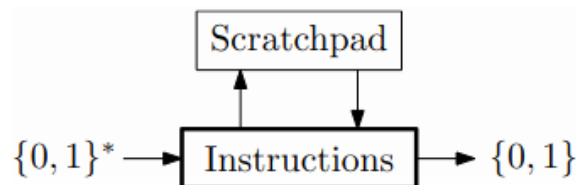
In order to prove our results we need a **computational model** in the form of an abstract machine, relying on two main assumptions:

1. It should be much **simple** as possible.
2. It must be able to run any other type of machine.

Our reference model will be the **Turing Machine**, that guarantees both the previous peculiarities.

### 2.1 The model informally

An easy representation about the Turing Machine could be as follows.



where:

- $\{0, 1\}^* \rightarrow$  alphabet that provides us a whatever string of any dimension
- *Scratchpad*  $\rightarrow$  physical part of the Turing Machine
- *Instructions*  $\rightarrow$  set of symbols used by the Turing Machine
- $\{0, 1\} \rightarrow$  answer of the Turing Machine as a single character

Therefore, given in input a whatever string from the alphabet  $\{0, 1\}^*$  returns 0 or 1, that respectively described when a **decision problem** is **decidable** or **undecid-**

**able.** Informally, if we want to compute a function  $f : \{0, 1\}^* \rightarrow \{0, 1\}$ , we need a finite, simple and generic set of instructions. Instructions of the Turing Machine are divided into:

1. Read a bit.
2. Read a symbol from the scratchpad.
3. Write a symbol into the scratchpad.
4. Declare the computation finished or terminated.

The **running time** of any Turing Machine on the input  $x$  is simply the number of basic instructions made. Generally, we say that the machine **runs in time**  $T(n)$  if it performs at most  $T(n)$  instructions on the input string of length  $n$ . Notice, every instruction made by the Turing Machine is done for each bit or *cell* that composed the original string  $x$  given in input.

Since the set of instructions is finite, the instructions can be encoded as strings, and therefore the machine can be described as a string.

#### Definition

Given a string  $\alpha$ , we indicate with  $M_\alpha$  the Turing Machine encoded by  $\alpha$ .

#### Definition

With  $U$  we indicate the **Universal Turing Machine**, which simulates any other Turing Machine given its string representation (*as  $\alpha$  in the previous definition*): given the pair  $(x, \alpha)$  the machine  $U$  simulates the behaviour of  $M_\alpha$  on  $x$ , where:

- $\alpha \rightarrow$  encoding of the Turing Machine  $M_\alpha$
- $x \rightarrow$  input string of whenever dimension

However, there are functions as  $f : \{0, 1\}^* \rightarrow \{0, 1\}$  which are intrinsically uncomputable by Turing Machines. We need a more formal way in order to prove their **uncomputability**.

## 2.2 The model formally

Any Turing Machine is composed by a **scratchpad** and a set of **instructions**. More formally:

**1<sup>st</sup> Scratchpad.**

The scratchpad consists of  $k$  **tapes** ( $k \geq 3$ ), each of them is a infinite one-directional

sequence of cells. Each cell stores a single symbol from a finite alphabet  $\Gamma$ , also called the alphabet of the machine. Every tape has also a **head**, capable of reading or writing one symbol at a time. These tapes are categorized into:

1. **Input tape**: typically read-only data structures, these store the initial input string  $x$ .
2. **Work tape**: intermediate tapes used for the computation of the input string  $x$ .
3. **Output tape**: tape that contains the result of the computation.

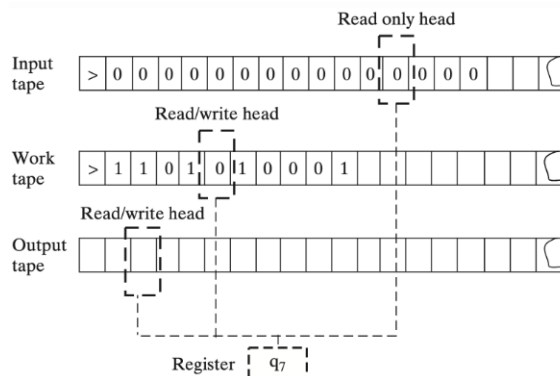
The tape alphabet  $\Gamma$  includes the following essential symbols:

- **Blank** symbol  $\square$ , indicates an empty cell.
- **Start** symbol  $\triangleright$ , marks the beginning of a tape.
- **0** and **1** symbols, since we are using strings.

## 2<sup>nd</sup> Instructions.

The machine operates through a finite set of states,  $Q$ . During each computational step, the machine performs the following sequence of actions:

1. **Read** the  $k$  symbols under the  $k$  tape heads.
2. For the  $k - 1$  read-write tapes, **replace** the current symbol with a new one, or leave it unchanged.
3. **Change** its state to a new one.
4. **Move** each of the  $k$  tape heads to the left, to the right or stay in place.



With these components defined, we have the necessary framework to construct a formal definition of any Turing Machine.

### Definition

A **Turing Machine** working with  $k$  tapes is described by the triple  $(\Gamma, Q, \delta)$ , where:

- $\Gamma \rightarrow$  tapes' alphabet.
- $Q \rightarrow$  set of states, which includes an **initial state**  $q_{init}$  and a **final state**  $q_{halt}$ .
- $\delta \rightarrow$  **transition function**

$$\delta : Q \times \Gamma^k \rightarrow Q \times \Gamma^{k-1} \times \{L, S, R\}^{k-1}$$

The transition function maps the machine's current configuration to its next configuration, defined as:

- $(Q \times \Gamma^k) \rightarrow$  the machine considers its current state  $q \in Q$  and the  $k$  symbols currently read by the tape heads.
- $(Q \times \Gamma^{k-1} \times \{L, S, R\}^{k-1})$ :
  - \*  $Q \rightarrow$  the machine shifts to a new state.
  - \*  $\Gamma^{k-1} \rightarrow$  the machine updates the symbols of  $k - 1$  read-write tapes, since the input tape is read-only.
  - \*  $\{L, S, R\}^k \rightarrow$  the machine specifies a direction (*Left*, *Stay*, *Right*) for each of the  $k$  tape heads.

## 2.3 Machine configurations

Another important notion about any Turing Machine, described by the triple  $(\Gamma, Q, \delta)$ , is the **configuration**.

### Definition

A **configuration** consists of:

- The current state  $q$ .
- The contents of  $k$  tapes.
- The positions of  $k$  tape heads.

There exists only one special configuration among the others, named the **initial configuration**.

### Definition

The **initial configuration**  $I_x$  for the input  $x \in \{0, 1\}^*$  is the configuration in which:

- The current state is  $q_{init}$ .
- The first tape contains  $\triangleright x$ , followed by blank symbols  $\square$ , while the other tapes contain  $\triangleright$ , followed by blank symbols  $\square$ .
- The tape heads are positioned on the first symbol of the  $k$  tapes.

Additionally, there's also a **final configuration** that does not require any particular property as before.

### Definition

A **final configuration** for the output  $y \in \{0, 1\}^*$  is any configuration of the Turing Machine whose state is  $q_{halt}$ , in which the content of the output tape is  $\triangleright y$ , followed by blank symbols  $\square$ .

## 2.4 Machine computations

Let's consider a Turing Machine described by the triple  $M = (\Gamma, Q, \delta)$  working on  $k$  tapes. The *transition function*  $\delta$  must determine in a natural way the next configuration of the current Turing Machine.

### Definition

Given any starting configuration  $C$ , the transition function  $\delta$  determines the next configuration  $D$ , and we write

$$C \xrightarrow{\delta} D$$

if this is the case.

Any machine will return an output working on a given input in a total amount of steps.

### Definition

$M$  returns  $y \in \{0, 1\}^*$  on input  $x \in \{0, 1\}^*$  in  $t$  steps if

$$I_x \xrightarrow{\delta} C_1 \xrightarrow{\delta} C_2 \xrightarrow{\delta} \dots C_t$$

where  $C_t$  is a final configuration for  $y$ . We write  $M(x)$  for  $y$  if it holds.

### Definition

$M$  computes a function  $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$  if and only if

$$M(x) = f(x) \text{ for every } x \in \{0, 1\}^*.$$

In this case,  $f$  is said to be **computable**.

## 2.5 Efficiency and runtime

Solving a function can be done by a **simulation** of a Turing Machine and we can design several machines for the resolution of the same computational task. We need a way to compare them and generally it's done counting how many **transitions** are required to achieve the output  $y$ .

### Definition

A Turing Machine  $M$  computes a function  $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$  **in time**  $T : N \rightarrow N$  if and only if  $M$  returns  $f(x)$  on input  $x$  in a number of steps smaller or equal to  $T(|x|)$  for every  $x \in \{0, 1\}^*$ . In this case,  $f$  is said to be **computable** in time  $T$ .

### Definition

A language  $L_f \subseteq \{0, 1\}^*$  is **decidable** in time  $T$  if and only if  $f$  is computable in time  $T$ .

Note:  $L_f$  is a subset of alphabet  $\{0, 1\}^*$  mainly used by the characteristic function.

In this case we do not have a real difference between **computable** and **decidable**, they can be used as synonyms.

## 2.6 Machines as strings

We are moving slowly to the main topic of this section: **everything** is not computable.

The main advantage of keeping our definition about a Turing Machine simple as much as possible is that any machine described by the triple  $M = (\Gamma, Q, \delta)$  can be determined by a subset of  $\delta$ , more formally:

$$\delta : Q \times \Gamma^k \times Q \times \Gamma^{k-1} \times \{L, S, R\}^k$$

where  $\delta$  can be mapped as a graph. This means that by any acceptable encoding of  $\delta$  as binary string in  $\{0, 1\}^*$  defines a valid encoding  $[M]$  for the current Turing Machine  $M$  used. By the way, the representation of the Turing Machine by  $\delta$  is possible providing the following conditions:

1. Every string in  $\{0, 1\}^*$  represents a Turing Machine.
2. Every Turing Machine  $M$  is represented by an infinitely many strings.

## 2.7 Universal Turing Machine

### Theorem

There exists a Turing Machine  $U$  such that for every  $x, \alpha \in \{0, 1\}^*$ , it holds that  $U(x, \alpha) = M_\alpha(x)$ , where  $M_\alpha$  denotes the Turing Machine represented by  $\alpha$ . Moreover, if  $M_\alpha$  halts on input  $x$  within  $T$  steps then  $U(x, \alpha)$  halts within  $C \cdot T \cdot \log(T)$  steps, where  $C$  is independent of  $|x|$  and depending only on  $M_\alpha$ .

## 2.8 Uncomputability

### Theorem

There exists a function  $uc : \{0, 1\}^* \rightarrow \{0, 1\}^*$  that is not computable by any Turing Machine.

### Example

e.g. Let's consider the following function.

$$uc(\alpha) = \begin{cases} 0 & \text{if } M_\alpha(\alpha) = 1 \\ 1 & \text{otherwise} \end{cases}$$

We can prove that does not exist a Turing Machine to solve this function by **contradiction**. Assuming that exists a Turing Machine  $M$  such that  $M(\alpha) =$

$uc(\alpha) \forall \alpha \in \{0, 1\}^*$ , and in particular when  $\alpha = \lfloor M \rfloor$  we get:

$$uc(\lfloor M \rfloor) = 1 \iff M(\lfloor M \rfloor) \neq 1 \iff uc(\lfloor M \rfloor) = 0$$

which is obviously a contradiction, so  $uc$  is not computable by any Turing Machine.

Notice: the resolution of any function is done by the following equivalence:  $M(x) = f(x)$  for every  $x \in \{0, 1\}^*$ . Simply, we are stating that to solve the function it must be the same thing as solving it by a Turing Machine. The example above does not work by construction, we are returning different outputs giving the same input, breaking the previous equivalence done.

## 2.9 The halting problem

The function  $uc$  we proved uncomputable does not represent an interesting computational task. We consider, instead, the function  $halt$  defined as follows:

### Definition

$$halt(\lfloor (\alpha, x) \rfloor) = \begin{cases} 1 & \text{if } M_\alpha \text{ halt on input } x \\ 0 & \text{otherwise} \end{cases}$$

where:

- $\alpha \rightarrow$  the encoding string
- $x \rightarrow$  input data

Being able to compute the  $halt$  function would mean being able to check algorithms for termination.

### Theorem

The function  $halt$  is not computable by any Turing Machine.



## 2.10 Rice's theorem

### Theorem

Every semantic decidable language is trivial, i.e. either  $L = \emptyset$  or  $L = \{0, 1\}^*$ .

A language  $L \subseteq \{0, 1\}^*$  is **semantic** when the following condition holds:

- $M$  and  $N$  compute the same function.
- The encoding strings of the machines belong to  $L$ ,  $\llbracket M \rrbracket \in L$  and  $\llbracket N \rrbracket \in L$ .

In other words, we are saying that two machines compute the same function if both their encoding strings belong to the same semantic language  $L$ .

# Chapter 3

## Polynomial time computable problems

### 3.1 Complexity classes

A **complexity class** is a set of *tasks* which can be computed within a specific quantity of resources (roughly speaking, these tasks are sharing the same complexity, how complex they are to solve).

Note: a **computational task** is what we really want to achieve and it can be expressed as a function or a decision problem.

Typically, the tasks we are interested in are decision problems, or equivalently languages (for instance, a subset of the alphabet  $\{0, 1\}^*$ ). In particular, let  $T : N \rightarrow N$  be a function used to detect how much time it's required for solving an instance of whatever problem, we state the following definition.

#### Definition

A language  $L$ , subset of  $S^*$  ( $L \in \{0, 1\}^*$ ), is in the class **DTIME**( $T(n)$ ) if and only if there is a *Turing Machine* deciding  $L$  and running in time  $n \rightarrow c \cdot T(n)$  for some constant  $c$ .

With the above definition we are saying that there exist some *Turing Machines* capable of solving the given task in linear time. This is not an universal definition, maybe other Turing Machines could require less time for the resolution. However, since a particular task has a time complexity **proportional** to  $T(n)$  we know for sure that even in the worst case it requires no more than linear time.

The letter **D** in **DTIME**(.) refers to *determinism*: the machines on which the class is based work deterministically, swapping to an already known new configuration  $C_i$ .

## 3.2 The class P

The complexity class at the bottom of our hierarchy is **P**. The class **P** is defined as follows:

$$P = \bigcup_{c \geq 1} \mathbf{DTIME}(n^c)$$

We get the union of all the **DTIME**( $n^c$ ) classes whose function is any polynomial time representation  $n^c$ , therefore obtaining an unique set of all those tasks sharing the same time complexity.

In other words, the class **P** includes all those languages  $L$  (decision problems) that can be decided (solved) by a Turing Machine in a running time  $P$ , where  $P$  can be any polynomial time representation of the form  $n^c$ .

More formally, for any polynomial **P** there are  $c, d$  natural numbers that are strictly greater than 0 such that  $P(n) \leq c \cdot n^d$  for sufficiently large  $n$ . By this last statement, we can observe that  $c$  and  $d$  can be arbitraly large or huge, so, for instance, a *Turing Machine* deciding a given language  $L$  (decision problem) in time  $10^{20} \cdot n^{10^{30}}$  still always a resolution in a polynomial time!

**P** is generally considered as **the class** of efficiently decidable languages, problems or tasks of this class seem to be simple, but anyway it's always possible to build a machine that solves the same problem or task with a different complexity.

## 3.3 The Strong Church-Turing Thesis

### The Church-Turing Thesis

- Every physically realizable computer can be simulated by a Turing Machine with a overhead in time.
- The class of computable tasks would not be larger if formalized in a realistic way, but differently.

### The Strong Church-Turing Thesis

- Every physically realizable computer can be simulated by a Turing Machine with **polynomial** overhead in time ( $n$  steps on the computer requires  $n^c$  on Turing Machines, where  $c$  only depends on the computer), and viceversa.
- The class **P** would be **the same** if defined based on other realistic models of computation.

Note: by the second thesis we specify that for any task belonging to the complexity class **P** its resolution by any Turing Machine always requires a *polynomial* overhead in time.

## 3.4 Why polynomials?

Why do we take the complexity class  $P$  as our main basis? There are three main reasons to do so, described as follows:

1. **P is robust**

Polynomials seem to be the smallest class of bounds which makes  $P$  robust.

2. **Exponents are often small**

The exponent  $c$  bounding the time of any Turing Machine capable to decide (solve) a language  $L$  (decision problem) such that  $L \in \mathbf{P}$  can be huge. Anyway, in almost of the cases, the computational tasks solved have a complexity not greater than quadratic or cubic.

3. **Nice closure properties**

It is relatively easy to prove that a given problem or task is **in** the class. Without constructing a Turing Machine explicitly, we can design an algorithm that guarantees the resolution of the problem in a polynomial running time.

## 3.5 The complexity class FP

Sometimes, one would like to classify **functions** rather than **languages** (remember, a computational task can be expressed by a function or a language). This can be done generalizing some concepts we have previously introduced.

In particular, let  $T : N \rightarrow N$  be the function used to detect how much time it's required for solving an instance of whatever problem, we state the following definition.

### Definition

A function  $f$  is in the class **FDTIME**( $T(n)$ ) if and only if **there is** a Turing Machine computing  $f$  and running in time  $n \rightarrow c \cdot T(n)$  for some constant  $c$ .

The complexity class **FP** is defined as below, very similar to the complexity class **P**:

$$\mathbf{FP} = \bigcup_{c \geq 1} \mathbf{FDTIME}(n^c)$$

From the definition of the complexity class **FP** we can also describe other two peculiarities:

- For every language  $L \in P$ , the characteristic function  $f$  of  $L$  is trivially in **FP**.
- For certain classes of functions, there are canonical ways to turn a function  $f$  into a language  $L_f$ . However, it is not true that  $f \in \mathbf{FP}$  implies  $L_f \in \mathbf{P}$ .

## 3.6 How to prove a task being in P or FP

In theory, we would design a Turing Machine, working within some polynomial boundaries, and prove that this machine decides (solves) the language (decision problem) or computes the function. However, this approach will take ages.

Instead of going through Turing Machines, we could use an informal but more expressive tool the so called **pseudocode**.

### Example

e.g. Suppose we want to show the following problem to be computable in polynomial time:

*given two strings  $x, y \in \{0, 1\}^*$  determine if  $x$  contains an instance of  $y$  (is  $y$  a substring of  $x$ ?)*

A pseudocode solving the problem above is the following.

```
i ← 1;
while i ≤ |x| - |y| + 1 do
  if x[i : i + |y| - 1] = y then
    | return True
  else
    | i ← i + 1
  end
end
return False
```

The most amount of work is done inside the *if condition*: by pseudocode we

cannot anymore state that every operation has the same unary cost as Turing Machines. In this case the amount of work done is at least equal to the size of  $y$ .

How could we be sure that the algorithm shown works in polynomial time? We must satisfy some constraints about that, divided into:

1. The input can be easily encoded as a binary string. Notice, we use pseudocode as a short-cut, but the formal proof should be done by the construction of a Turing Machine.
2. The total number of instructions executed is polynomially bounded.
3. All intermediate results are polynomially bounded in length.
4. Each instruction takes polynomial time to be simulated.

#### Example

e.g. Defining the polynomial complexity of the previous algorithm.

1. The total number of instructions executed is bounded by  $O(|x|)$ .
2. The intermediate results obtained over  $i$  cannot be greater than  $O(|x|)$ .
3. Comparing two strings of length  $|y|$  can be done in polynomial time in  $|y|$ , thus polynomial in  $||x, y||$

### 3.7 The class EXP

The next class of functions  $T : N \rightarrow N$ , beyond the polynomials and having nice closure properties is the class of exponential functions. The complexity classes **EXP** and **FEXP** are defined as follows:

$$\mathbf{EXP} = \bigcup_{c \geq 1} \mathbf{DTIME}(2^{n^c}) \quad \mathbf{FEXP} = \bigcup_{c \geq 1} \mathbf{FDTIME}(2^{n^c})$$

The tasks in these classes can be solved mechanically, but possibly cannot be solved efficiently. Of course, the previous complexity classes **P** and **FP**, respectively for languages and functions, are subsets of the ones introduced.

$$\mathbf{P} \subseteq \mathbf{EXP} \quad \mathbf{FP} \subseteq \mathbf{FEXP}$$

#### Theorem

The two inclusions above are strict.

The theorem seems to be quite weird, but what it's saying is quite easy to understand:

- Some problems are in **EXP** complexity class but not in **P**.
- Some functions are in **FEXP** complexity class but not in **FP**.

## Chapter 4

# Between the feasible and the unfeasible

We are going to introduce a new big topic not seen so far, located between feasible and unfeasible problems, or in other words between  $P$  and  $EXP$  complexities.

### 4.1 The border between the tractable and the intractable

Up to now, we have seen two types of complexity classes, namely:

- **P**, class of easy problems or **tractable** problems, that can be solved in polynomial time, even though the polynomial number is huge.
- **EXP**, which contains the whole class  $P$ , but also some problems which cannot be solved in polynomial time, intrinsically requiring exponential time, named **intractable** problems.

It's now time to study the **border** of these two classes, between tractable and intractable problems. The border is a great area of classes between  $P$  and  $EXP$ , so there isn't only the complexity class **NP**.

### 4.2 Creating v.s. Verifying

The first such class of this great area is **NP**.  $NP$  has to deal with a process called **verification**. Therefore, very often, the language we would like to classify can be



written as follows:

$$L = \{x \in \{0, 1\}^* \mid \exists y \in \{0, 1\}^{p(|x|)}. (x, y) \in A\}$$

where:

- $p : N \mapsto N \rightarrow$  polynomial function
- $A \rightarrow$  set of pairs of strings

This definition of language may seem quite tricky, but simply it says: the elements of language  $L$  are those strings for which we can find a **certificate**  $y$  (*of polynomial length*) such that the pair  $(x, y)$  passes the **test**  $A \subseteq \{0, 1\}^* \times \{0, 1\}^*$ .

A certificate  $y$  is a string of polynomial length, it cannot be too big compared to the string  $x$ . Therefore, in order to prove that  $x$  belongs to  $L$  through  $y$  must be a polynomial relation between their lengths.

The key point of this definition is the **existential quantifier**:  $x$  is precisely those strings such that there exist a **certificate**  $y$ . Additionally,  $A$  is a set that is really easy to check, what is called the **verifier**: verifying whether a given certificate is a good certificate for a string  $x$  is quite trivial. However, it's not necessary easy to check if such certificate exists or not.

Supposing there is a language  $L$  that can be expressed as the definition shown previously, such that the inner set of pairs of strings can be decided in polynomial time. **Does this imply that  $L$  is itself decidable in polynomial time?**

The real answer of this double implication question is **not necessarily**: given  $x$ , we can check whether  $x \in L$  by checking whether the couple  $(x, y) \in A$  for all possible  $y$  such that  $|y| \leq p(|x|)$ , of which however there are exponentially many.

The **take-away message** is thus the following: **crafting** a solution for the problem  $x$  (*finding*  $y$ ) can potentially be more difficult than just **checking**  $y$  to be a solution to  $x$ .

## 4.3 The complexity class NP

The class **NP** is defined in the following way:

### Definition

A language  $L \in \{0, 1\}^*$  is in the class **NP** if and only if there exist a polynomial  $p : N \rightarrow N$  and a polynomial time Turing Machine  $M$  such that:

$$L = \{x \in \{0, 1\}^* | \exists y \in \{0, 1\}^{p(|x|)}. M(\lfloor (x, y) \rfloor) = 1\}$$

where:

- $M \rightarrow$  **verifier** for language  $L$
- $y \in \{0, 1\}^{p(|x|)}$  s.t.  $M(\lfloor (x, y) \rfloor) = 1 \rightarrow$  **certificate** for  $x$

$NP$  is the class of problems such that certificates are polynomial in length and they can be verified in polynomial time.

Differently from  $P$  and  $EXP$  that have as counterpart the class of functions, the class  $NP$  does not have a natural counterpart as a class of functions.

Finally, we can state that  $NP$  sits between  $P$  and  $EXP$ . More formally:

### Theorem

$$P \subseteq NP \subseteq EXP$$

Every problem in **P** can be seen as a very special case of problem in **NP**, where the certificate  $y$  is empty. Therefore, for every decision problem  $L \in P$ , there is a Turing Machine  $N$  solving  $L$  in polynomial time, without the need of any certificate.

Formally, what we have discussed can be described as follows:

### Definition

If  $M$  is a machine that on output  $(\lfloor x, y \rfloor)$ , accepts it if and only if  $N$  accepts  $y$ , then:

$$L = \{x \in \{0, 1\}^* | \exists y \in \{0, 1\}^{p(|x|)}. M(\lfloor x, y \rfloor) = 1\}$$

where:  $p(|x|) = 0$ . So,  $y$  does not really exist.

## 4.4 Examples problems in NP

### Minimum independent set

e.g. We consider this task as a set of problems.

**Problem.**

It asks whether a pair  $(G, k)$  of an undirected graph and a natural number  $k \in \mathbb{N}$  is such that  $G = (V, E)$  admits an independent set  $W \subseteq V$  of cardinality at least  $k$ .

**Note.**

We are not computing the parameter  $k$  but, instead, we provide it as an input parameter. We use it in order to check if it exists an independent set of nodes of cardinality at least equal to  $k$ .

**Certificate.**

In this problem or language, the certificate coincides with the independent set  $W$ . Check its cardinality and independence is quite easy, but what is really tricky of this task is to find out the independent set  $W$ .

Formally, the certificate is described as follows:

$$L = \{(G, k) | \exists W . W \text{ is independent for } (G, k) \wedge |W| \geq k \wedge W \subseteq V\}$$

### Subset sum

e.g. Subset sum problem.

**Problem.**

It asks whether, given a sequence of natural numbers  $n_1, \dots, n_m$  and a number  $k$ , there exists a subset  $I$  of  $\{1, \dots, m\}$  such that:

$$\sum_{i \in I} n_i = k$$

**Certificate.**

It's pretty simple to identify the certificate of this language: the certificate here is just  $I$ , checking whether the summation is equal to a natural number is very easy.

In this problem, checking if the summation of natural numbers is equal to  $k$  is quite easy, but what is really tricky is to find out the subset of natural numbers  $I$ .

## Composite Numbers

e.g. Composite numbers problem.

### Problem.

Given a number  $n \in N$ , determine whether  $n$  is composite, not prime.

### Note.

In order to determine whether a number is composite or not, we don't need to take all its range of numbers, we can just use the  $\sqrt{n}$ . Anyway, even though by this consideration we have reduced the range of numbers, we have to check any of them.

For sure, checking every single number up to  $\sqrt{n}$  cannot be solved in polynomial time: the total amount of steps is exponential in the size of the input, which is  $\sqrt{n}$ .

### Certificate.

It is the factorization of  $n$ , a couple of natural numbers  $(m, l)$  such that  $n = m \cdot l$ .

More in details, the verifier and the certificate are respectively:

- Verifier, product between the two natural numbers,  $n = m \cdot l$ .
- Certificate, decomposition of  $n$  into a pair of natural numbers.

## Decisional Linear Programming

e.g. Decisional linear programming problem.

### Problem.

Given a sequence of  $m$  linear inequalities with rational coefficients over  $n$  variables, decide whether there is a rational assignment to the variables  $x_1, \dots, x_n$  which makes the inequalities true.

Inequalities are of the form:

$$\sum_{i=1}^n a_i x_i \leq b$$

where:

- $a_i \rightarrow$  rational coefficient
- $b_i \rightarrow$  rational number

### Note.

As usual, in one hand checking the satisfiability of these inequalities is trivial.

In the other hand, finding the  $n$  different rational numbers for the  $x_1, \dots, x_n$  variables is difficult.

**Certificate.**

The assignments, which can be easily checked for correctness.

Additionally, the assignments must be small enough, since the inequalities are linear. In the formal definition, this means that the binary encodings of the assignments taken in input by the Turing Machine cannot be too big.

### Decisional 0/1 Linear Programming

e.g. Decisional 0/1 linear programming problem.

**Problem.**

Given a sequence of  $m$  linear inequalities, decide whether there is an assignment of zeros and ones to the variables  $x_1, \dots, x_n$  making the inequalities true.

**Note.**

We have just reduced the search space of the assignments, so the certificate still be the same as in the example above. However, making this problem more "easy", we cannot anymore exploit some geometric properties of the inequalities to find out the best matches.

**Certificate.**

Again, the assignments.

Some of the discussed problems are also in **P**:

1. Composite numbers.
2. Decisional linear programming.

All the other problems, as minimum independent set, subset sum and decisional 0/1 linear programming, are currently **not known** to be in **P**: this does not mean that they cannot be solved, but up to now does not exist any algorithm that can solve them in polynomial time.

## 4.5 Nondeterministic Turing Machines

Sometimes the complexity class **NP** is defined by **nondeterministic Turing Machines**, rather than by certificates and verifiers.

A nondeterministic Turing Machine is a type of machine that can evolve in multiple ways, even though the configuration  $C_i$  still be the same. This new kind of machine does not include anymore algorithms, since they are not deterministic by structure.

The only differences between a **NDTM** and an ordinary **Turing Machine** are:

1. In a nondeterministic Turing Machine we don't have anymore a single transition function  $\delta_i$ , but two transition functions  $\delta_0$  and  $\delta_1$ . At each step, the machine picks one of the transitions in a complete nondeterministic way and proceeds according to it.
2. A special state named  $q_{accept}$ .

As for ordinary Turing Machines, we say that a nondeterministic Turing Machine  $M$ :

- **accepts** a binary string  $x \in \{0, 1\}^*$  if and only if there exists one among the possible evolutions of the machine  $M$  such that giving it  $x$  it reaches the state  $q_{accept}$ ;
- **rejects** a binary string  $x \in \{0, 1\}^*$  if and only if none of the possible evolutions of the machine  $M$  leads to  $q_{accept}$  when given in input  $x$ .

## 4.6 Time-Bounded NDTMs

We are gonna to describe something more formal about **nondeterministic Turing Machines**, in particular:

### Definition

We say that a NDTM  $M$  runs in time  $T : N \rightarrow N$  if and only if for every  $x \in \{0, 1\}^*$  and for every possible nondeterministic evolution,  $M$  reaches either the **halting state**  $q_{accept}$  within  $c \cdot T(|x|)$  steps, where  $c > 0$ .

### Definition

For every function  $T : N \rightarrow N$  and  $L \subseteq \{0, 1\}^*$ , we say that  $L \in \text{NDTIME}(T(n))$  if and only if there is a NDTM  $M$  working in time  $T$  and such that  $M(x) = 1$  if and only if  $x \in L$ .

### Theorem

$$\mathbf{NP} = \bigcup_{c \in \mathbb{N}} \mathbf{NDTIME}(n^c)$$

Therefore, we can define **NP** by nondeterministic Turing Machines: the definition based on certificates and verifiers is equivalent to the one based on NDTM. In particular, the two definitions are equivalent for the following reasons:

- The existential quantifiers express the same thing: one used for the  $q_{accept}$  state and the other for the certificate  $y$ .
- Looking for the right evolution of the nondeterministic Turing Machine is the same as looking for the existence of a certificate  $y$  polynomially bounded.

## 4.7 Are all problems in NP equivalent?

The answer to the question suggested by the title is **no**: all the problems in NP are not equivalent. For instance, the following two problems are not equivalent, even though they are both in NP:

1. Maximum independent set, a problem that we already know to be NP.
2. The language of palindromes is easy to be proved in P, so it's also in NP.

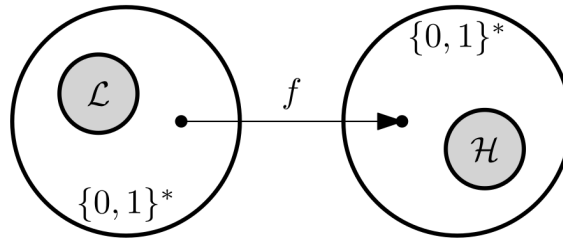
But, anyway, they are different: the degree of difficulty in proving these problems in NP is different. **Therefore, how can we distinguish problems in NP that have different degrees of difficulty?**

For sure, we cannot do so much knowing that a decision problem  $L$  is in NP, we need something else. However, we can look to the complexity class NP as a tool to study the relative difficulty between distinct problems. The relative difficulty between problems is expressed by a **pre-order relation** between languages.

## 4.8 Reductions

The comparison between problems' difficulties is done by **reductions**, something that we have already seen in the first chapters.

We would like to define a reduction between two decision problems (*namely languages*) through a function  $f$ . Let's consider the following image.



The function  $f$  takes in input a point from the left-most set and maps it to a point in the right-most set: considering a starting point inside the language  $L$ , the function  $f$  will land it inside the language  $H$ . Automatically we can say that  $L$  belongs to the same complexity of  $H$ .

### Definition

The language  $L$  is said to be **polynomial-time reducible** to another language  $H$  if and only if there is a polytime computable function  $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$  such that  $x \in L$  if and only if  $f(x) \in H$ .

If we are able to put in relation such decision problems by a polytime reduction, then we can say that  $L$ 's difficulty is at most equal to  $H$ 's difficulty: in order to express such relationship between decision problems, we use this unique notation described by a pedix  $p$ , meaning a polytime reduction.

Note: in the example above, the language  $L$  is difficult as  $H$  and it cannot be more difficult.

## 4.9 Reductions and Complexity

If  $L$  has the same complexity of  $H$ , namely  $L \leq_p H$ , then  $H$  is at least as hard as  $L$ , at least as far as classes like  $\mathbf{P}$  are concerned.

The previous statement simply says: if  $L \leq_p H$  and  $H \in \mathbf{P}$ , then also  $L \in \mathbf{P}$ . A possible method to solve  $L$  in this case is to take an instance of  $L$  and translate it into an instance of  $H$ , namely  $f(x)$ , which must be done in polynomial time: checking whether  $f(x) \in H$  gives the answer to the original question,  $x \in L$ .

Suppose we have a problem  $H \subseteq \{0, 1\}^*$ . It's said to be:

- **NP-hard** if  $L \leq_p H$  for every language  $L \in \mathbf{NP}$ .
- **NP-complete** if  $H$  is **NP-hard**, and  $H \in \mathbf{NP}$ .



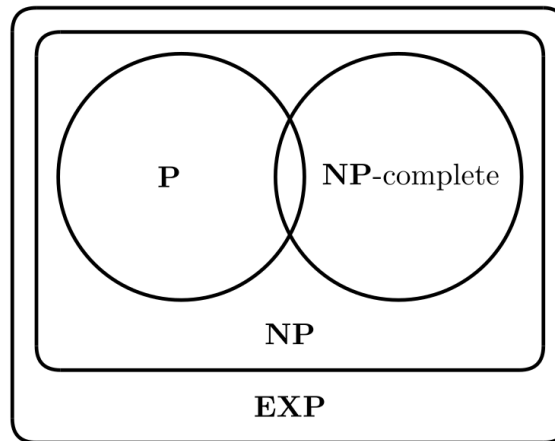
More in details, **NP**-hard defines all the problems that have a difficulty greater than any other language  $L$  inside **NP**. However, this does not mean that  $H$  is in **NP**, we are just stating that the problem  $H$  is more difficult than any other problem in **NP**.

In the other hand, **NP**-complete defines all the problems  $H$  already in **NP** that are more difficult than any other problem in **NP**.

### Theorem

1. The relation  $\leq_p$  is a pre-order (*i.e. it is reflexive and transitive, meaning that any language  $L$  is at least as difficult as the language itself and we can form chains of reductions to find out the complexity of a given decision problem*).
2. If  $L$  is **NP**-hard and  $L \in \mathbf{P}$ , then  $\mathbf{P} = \mathbf{NP}$ .
3. If  $L$  is **NP**-complete, then  $L \in \mathbf{P}$  if and only if  $\mathbf{P} = \mathbf{NP}$ .

Let's take a look at the following representation.



From the picture above, we get how the complexities are related between each other, in particular:

$$\mathbf{P} \subseteq \mathbf{NP} \subseteq \mathbf{EXP}$$

The intersection between  $\mathbf{P}$  and **NP**-complete is an empty area: they are completely disjoint. Therefore,  $\mathbf{P}$  and **NP**-complete can be seen as extremes of the complexity class **NP**:  $\mathbf{P}$  is the easiest extreme while **NP**-complete is the hardest extreme.

## 4.10 The First Example of an NP-complete Problem

One obvious way of building an **NP**-complete problem is to define it as the problem of **simulating** any Turing Machine.

Let *TMSAT*, acronym of Turing Machine Satisfiability, be the following language:

$$TMSAT = \{(\alpha, x, 1^n, 1^t) \mid \exists u \in 0, 1^n . M_\alpha \text{ outputs } 1 \text{ on input } (x, u) \text{ within } t \text{ steps}\}$$

where:

- $\alpha \rightarrow$  index of the Turing Machine
- $x \rightarrow$  input string
- $1^n \rightarrow$  size of the certificate
- $1^t \rightarrow$  time boundary

We are just asking if the machine  $\alpha$  returns 1 on input  $(x, u)$  within a total amount of  $t$  steps. The amount of steps  $t$  is not given in input since the Turing Machine would build solutions using also  $t$  as a piece of knowledge: doing so we avoid that the complexity grows exponentially in the size of  $t$ .

### Theorem

*TMSAT* is **NP**-complete.

## 4.11 A Quick Recap on Propositional Logic

**Propositional formulas** are the first problems to be proved in **NP**. Formulas in propositional logic are composed by:

- Propositional variables.
- Basic connectives  $\wedge, \vee$  and  $\neg$ .

A simple example could be:

$$X \vee \neg X, \quad X \wedge (Y \vee \neg Z)$$

Given a formula  $F$  and an assignment  $\rho$  of elements from  $\{0, 1\}$  to the propositional variables in  $F$ , we can define the **truth value** for  $F$ , indicated as  $[F]_\rho$ , by induction

on  $F$ :

$$\begin{aligned}[X]_\rho &= \rho(X) \\ [F \wedge G]_\rho &= [F]_\rho \cdot [G]_\rho \\ [F \vee G]_\rho &= [F]_\rho + [G]_\rho \\ [\neg F]_\rho &= 1 - [F]_\rho\end{aligned}$$

A formula  $F$  is **satisfiable** if and only if there is one assignment  $\rho$  such that  $[F]_\rho = 1$ .

## 4.12 The Cook-Levin Theorem

Searching for the satisfiability of this kind of decision problems based on propositional formulas, it's like searching for the decidability of them.

A propositional formula  $F$  is said to be in **conjunctive normal form** when it is a conjunction of disjunctions of literals: inner disjunctions and outside conjunctions. For instance, the following formulas are both CNFs:

$$X \vee \neg X, X \wedge (Y \vee \neg Z)$$

The disjunctions in a CNF are said to be **clauses**, and  $k$ **CNF** is a CNF whose clauses contains at most  $k$  literals, with  $k \in \mathbb{N}$ .

$$X \vee \neg X, X \wedge (Y \vee \neg Z)$$

Both of the propositional formulas above are 2CNFs.

### Theorem

The following two languages are **NP**-complete:

$$\begin{aligned} SAT &= \{[F] \mid F \text{ is a satisfiable CNF}\} \\ 3SAT &= \{[F] \mid F \text{ is a satisfiable 3CNF}\} \end{aligned}$$

## 4.13 Proving a Problem Hard

Suppose we are studying the complexity of a language  $L$ , and we are convinced that the given problem is **hard**. How should we prove our claim about the complexity or difficulty of the decision problem  $L$ ?

Prove that the language  $L$  belongs to any possible complexity class seen so far, like P, NP and EXP, does not mean so much. However, by proving that  $L$  is **NP**-complete is the best way to prove that the problem is not so hard, but also not so easy.

Proving that a given problem  $L$  is NP-complete it does not mean only that it's in NP but also is at least as difficult as any other problem in NP.

If we would like to prove that  $L$  is **NP**-complete, we have to prove two statements:

1.  $L$  is in **NP**.

We show that there are  $p$  polynomial and  $M$  polytime Turing Machine such that  $L$  can be written as:

$$L = \{x \in \{0, 1\}^* | \exists y \in \{0, 1\}^{p(|x|)} . M(x, y) = 1\}$$

2.  $H \leq_p L$  for any other language  $H$  in **NP**.

We take a specific language or decision problem already known to be in **NP** and we try to set up a reduction. In this step, we can exploit the transitive property of the reduction, building chains of reductions from known NP-complete problems to  $L$ .

## 4.14 The Zoo of NP-complete Problems

There's a zoo of NP-complete problems. For any pair  $(L, H)$  of **NP**-complete problems, we have that:

$$L \leq_p H \quad H \leq_p L$$

meaning that  $L$  and  $H$  are equivalent. By the mechanism of reductions, we have an easy way to show the equivalences between decision problems, even though for one of them does not exist a polytime algorithm solving it. Being said this, defining a reduction from  $L$  to  $H$  sometimes is easy, but sometimes is very difficult, and it is useful to think at **NP**-complete problems as a graph, where edges are natural polytime reductions.

As we already guessed, between **P** and **EXP** there is **NP**, a complexity class from which we can distinguish hard problems from complete problems.

## 4.15 ...

...

# Chapter 5

## The basics of Computational Learning Theory

### 5.1 Computational Learning Theory

Up to now, we have been considering negative results about **functions** as computational tasks. In particular, we defined a method to describe which tasks are easy or difficult to solve, studying the border between the **feasible** and the **unfeasible**.

Would it be possible to apply the same reasoning also for tasks coming from **machine learning**? Therefore, our purpose is to include inside what we have seen so far also functions related to machine learning tasks. We would like to understand which are the limits of machine learning, defining tasks achievable and not-achievable.

Formally, this kind of extension is called **computational learning theory** and it gives us answers to the following questions:

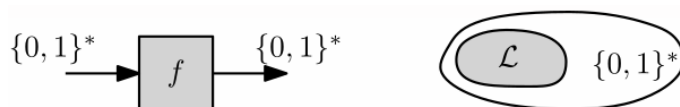
- Which functions are easy to learn?
- Are there functions which are instead hard to learn?
- How much data is required to learn a given function?
- Can we learn functions with limited computational resources?

### 5.2 PAC Learning

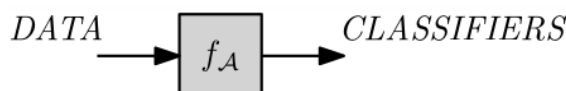
The first part of **computational learning theory** is about **PAC learning**, acronym of Probably Approximately Correct. The first two terms introduced **probably** and **approximately** are the main keys about this topic, let's see why.

## 5.3 Learning Problems as Computational Problems

Until now, we have so far taken a computational task as a function (*that takes input a string and returns another string*) or a language (*that takes in input a string and returns a boolean value*). Below, we have a picture about the two representations: the left-most is a function, while the right-most is a language.



However, can **learning problems** be seen as **computational problems**? The answer is positive: any learning algorithm  $A$  can be seen as an algorithm computing a function  $f_A$  whose input is a finite sequence of **labelled data** and whose output can be seen as a **classifier**.

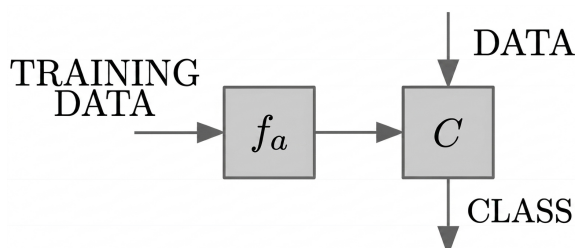


Conversely of what we did in **computation theory**, the output is not anymore a string or a boolean value but instead a **classifier**, in other words another function. This new type of output, namely a function, is a classifier able to categorize new data similar to the input labelled data.

The previous observations lead us to the following questions:

1. Could data and classifiers be encoded as strings, such that  $f_A$  becomes a function of the kind we already know?
2. How could we formalize the fact that  $A$  correctly solves a given learning task?

More in details, the discussion done until now can be figured as in the image below.



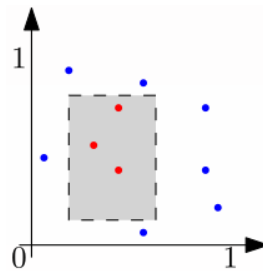
It's a kind of pipeline, divided into the following steps:

1. **Training data**, we get training data in order to **train** our model, in particular to recognize interesting patterns over the labelled data.
2.  $f_A$ , computing the function  $f_A$  we learn the relationship between data and labels.
3.  $C$ , output after the computation of the function  $f_A$ , which is itself a function, in particular a classifier.
4. **Data**, data coming in the classifier  $C$  are **not labelled data**.
5. **Class**, the real task of the classifier  $C$  is to find out an appropriate label of any **unlabelled data** given in input to it.

## 5.4 An Example Problem

### Example

e.g. An example of a learning problem to understand when the learning algorithm designed is correct or not.



Suppose that the data given to the algorithm  $A$  are pairs of real numbers, defined as follows:

$$(x, y) \in R_{[0,1]} \times R_{[0,1]}$$

where each coordinate has a value between 0 and 1. These are labelled as positive or negative depending on being inside or outside a **rectangle**.

We would like to learn a classifier that taking in input the whole set of points is able to infer the shape of the grey rectangle. However, the real shape of the final rectangle is unknown, it just knows the color of each point: therefore the learning algorithm should guess, trying several times, providing the rectangle as the output classifier  $C$ .

Note: until now we discussed about the computation of the function  $f_A$ . Remember, only after the computation of the function  $f_A$  we will get the classifier  $C$ .

By the way, the amount of training data could be potentially uncountable and infinite, so there is no hope that the output rectangle it's exactly the one that we are looking for.

The algorithm  $A$  cannot guess the exact rectangle  $R$  with perfect accuracy if the data received are too few. As the training data grows in number, we would expect the rectangle obtained by  $f_A(D)$  converges to  $R$ , wouldn't we? This is almost impossible, there would be always a little bit of freedom, making the deduced rectangle different from the exact one.

Instead of defining the correctness of a learning algorithm through this strict equivalence, we can study its behavior as it approaches to the **limit**.

## 5.5 The Rules of the Game

For sure,  $A$  knows or will learn that the way input data are labelled is by way of a rectangle: red points are inside the rectangle, while blue points are outside the rectangle. Anyway, at the same time, there are some things that it does not know, which are:

- It does not know which is the exact rectangle searched.
- It does not know the distribution  $\mathbf{D}$  from which the points  $(x, y)$  are drawn.

About the last statement, we don't want that our learning algorithm works only with few or specific distributions of data. We would like that  $A$  works well **independently** from any distribution from which we sample data. For these main reasons the distributions of data cannot be something that we can use to build our solution.

## 5.6 The Algorithm $A_{BFP}$

We could try to define an algorithm  $A_{BFP}$  to solve the previous problem, in particular:

1. Given the data  $((x_1, y_1), p_1), \dots, ((x_n, y_n), p_n)$ , where  $p_i$  defines the color of the point.
2. Determine the smallest rectangle  $R$  including all the positive instances, namely the red points.
3. Return the rectangle  $R$ .

Running such algorithm, the result will be always a sub-rectangle of the target rect-



angle. Since the output rectangle is not the one desired, is the algorithm proposed incorrect? The answer is **negative**: what we would like to understand is the behavior of the learning algorithm at the limit, while it is **converging** to the optimal rectangle.

For a given rectangle  $R$  and a target rectangle  $T$ , the **probability of error** in using  $R$  as replacement of  $T$  is:

$$\text{error}_{\mathbf{D},T}(R) = Pr_{x \sim \mathbf{D}}[x \in (R - T) \cup (T - R)]$$

where:

- $x \rightarrow$  input data coming from a unknown distribution
- $\mathbf{D} \rightarrow$  unknown distribution
- $R \rightarrow$  output rectangle
- $T \rightarrow$  target rectangle

Note: the target rectangle  $T$  is also called ground-truth in machine learning tasks.

Roughly speaking, the formula tells us: the error of using  $R$  as a replacement of  $T$ , when points are drawn from distribution  $\mathbf{D}$ , is equal to the probability that a randomly drawn point falls into the region where  $R$  and  $T$  disagree.

Note: the difference between the regions  $R$  and  $T$ , or vice-versa, is not the intersection of the two, but instead the **symmetric difference**. Therefore, the error counts only the points where the two rectangles disagree. Since  $R$  is always a sub-rectangle of  $T$ , the first difference,  $(R - T)$ , will be always the empty set,  $\emptyset$ .

As the number of samples inside the training set  $D$  grows, the result  $A_{BFP}(D)$  does not necessarily approach to the target rectangle (*since it's unfeasible and impossible*), but its probability of error converges to 0.

### Theorem

For every distribution  $\mathbf{D}$ , for every  $0 < \epsilon < \frac{1}{2}$  and for every  $0 < \delta < \frac{1}{2}$ , if  $m \geq \frac{4}{\epsilon} \ln(\frac{4}{\delta})$ , then:

$$Pr_{D \sim \mathbf{D}^m}[\text{error}_{\mathbf{D},T}(A_{BFP}(D)) < \epsilon] > 1 - \delta$$

This formula can be divided into:

- **Inner probability** ( $\text{error}_{\mathbf{D},T}(A_{BFP}(D)) < \epsilon$ ), the probability of an error defined by the output of the  $A_{BFP}$  algorithm in place of  $T$ , when points are drawn randomly from a distribution  $\mathbf{D}$ , is less then  $\epsilon$ .

- **Outer probability** ( $Pr_{D \sim \mathcal{D}^m}[\text{error}_{\mathcal{D},T}(A_{BFP}(D)) < \epsilon] > 1 - \delta$ ), the probability that the previous error is smaller than  $\epsilon$  is greater than  $1 - \delta$ .

Some interesting observations are already coming out:

- As soon as we take  $\epsilon$  and  $\delta$  close to 0, the number of training data grows exponentially, since these parameters occur as denominators while defining  $m$ .
- There are two probabilities related to the fact that data are drawn twice: one time for the training data and another time for the testing data. In this case, the inner probability is about the **accuracy** over the testing data, instead the outer probability is about the **confidence** over the training data.

We could suppose that for parameters  $\epsilon$  and  $\delta$  pretty close to 0 and for  $m$  enough big, we can get an error less than  $\epsilon$  and therefore the general probability will be very close to 1, describing the convergence of the learning algorithm to the optimal solution.

## 5.7 The General Model - Terminology

Let's see how we can generalize what we have discussed so far. First of all, we are gonna to describe some useful definitions, as follows.

### Instance Space $X$

We assume to work within an **instance space  $X$** :

- $X$  is the set of instances of objects the learner wants to classify.
- Data from the instance spaces are generated through a distribution  $\mathcal{D}$ .
- The instance space  $X$  is the set where training data and testing data belong.

Instance space can be seen as the set of **raw data**, that follows an unknown distribution  $\mathcal{D}$  and does not include labels at all.

Note: labelled data will be provided to our learning algorithm by an **oracle**, formally described as  $EX(c, \mathcal{D})$ .

### Concept

**Concepts** are subsets of  $X$ , thought as properties of data or objects. A concept is an arbitrary subset of the instance space  $X$ .

In the previous example about rectangles, concepts are arbitrary subsets of the instance space  $X = R_{[0,1]}^2$ , since we take pairs of coordinates. For instance:

- Rectangle has parallel axes.
- Red points are inside the rectangle.
- Blue points are outside the rectangle.

However, these concepts are not provided as input to the learning algorithm: it should derive these concepts while learning patterns from training data.

### Concept class

A **concept class**  $C$  is a collection of concepts, namely a subset of  $P(X)$ . These are the concepts we would like to learn during the training phase.

For instance, the concept class in the previous example that we learnt is the one of rectangles whose sides are parallel to the axes.

Note: the learning algorithm did not know that the classifier would be a rectangle whose sides are parallel to axes, only during the training phase it recognized this behavior from data.

The **target concept**, namely  $c \in C$ , is the concept, so the property, that the learner wants to build a classifier for (*boolean function to recognize when a point is inside or outside the rectangle*).

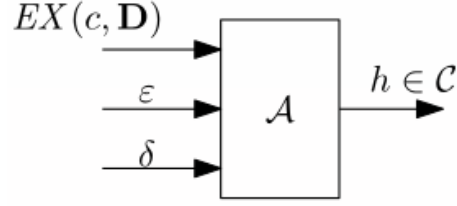
## 5.8 The General Model - The Learning Algorithm

### $A$

Every learning algorithm is designed to learn concepts from a concept class  $C$  but it does not know the target concept  $c \in C$ , and also the associated distribution  $\mathbf{D}$ .

We would like anyway that our learning algorithm is able to learn from any possible distribution: for that main reason the distribution  $\mathbf{D}$  of data cannot be known.

The interface of any learning algorithm  $A$  can be described as follows:



where:

- $EX(c, \mathbf{D}) \rightarrow$  should be thought as an **oracle**, a procedure that  $A$  can call as many times it wants, and which returns element  $x \sim \mathbf{D}$  from the instance space  $X$ , labelled according to whether it is in the target class  $c$  or not
- $\epsilon \rightarrow$  **error parameter**, it can be expressed also as the accuracy that we would like to achieve
- $\delta \rightarrow$  **confidence parameter**, the level of confidence should we achieve during the training

Therefore, the learning algorithm  $A$  should be able to perform the oracle  $EX(c, \mathbf{D})$  to get labelled data as many times it wants: during the training phase it should learn repetitive patterns from labelled data, giving us as final result a **hypothesis**  $h \in C$ . We would like a hypothesis  $h$  that is as similar to the target class  $c$  as possible.

The **error** of any hypothesis  $h \in C$  is defined as:

$$error_{\mathbf{D},c}(h) = \Pr_{x \sim \mathbf{D}}[h(x) \neq c(x)]$$

The **error** of a hypothesis  $h$  with the respect to a target concept  $c$  and a probability distribution  $\mathbf{D}$  is defined as the probability that  $h$  and  $c$  disagree on a randomly drawn point  $x$ .

## 5.9 The General Model - PAC Concept Classes

We can say that a machine learning task is **feasible** when the following definition is satisfied.

### Definition

Let  $C$  be a concept class over the instance space  $X$ . We say that  $C$  is **PAC learnable** if and only if exists an algorithm  $A$  such that for every  $c \in C$ , for every distribution  $\mathbf{D}$ , for every  $0 < \epsilon < \frac{1}{2}$  and for every  $0 < \delta < \frac{1}{2}$ , is such that:

$$\Pr[error_{\mathbf{D},c}(A(EX(c, \mathbf{D}), \epsilon, \delta)) < \epsilon] > 1 - \delta$$

As before, we can divide the previous formula into:

1. **Inner probability** ( $error_{\mathcal{D},c}(A(EX(c, \mathcal{D}), \epsilon, \delta))$ ), the probability of an error over the hypothesis  $h$  is less than  $\epsilon$ . Therefore,  $\epsilon$  is like an **upper bound** of the entire error probability.
2. **Outer probability** ( $\Pr[error_{\mathcal{D},c}(A(EX(c, \mathcal{D}), \epsilon, \delta)) < \epsilon] > 1 - \delta$ ), the probability that the previous error over the hypothesis  $h$  is smaller than  $\epsilon$  is greater than  $1 - \delta$ .

### Definition

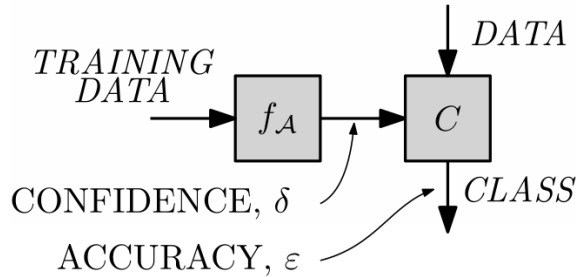
If the time complexity of  $A$  is bounded by a polynomial  $\frac{1}{\epsilon}$  and  $\frac{1}{\delta}$ , we say that  $C$  is **efficiently PAC learnable**.

### Corollary

The concept-class of axis-aligned rectangles over  $R_{[0,1]}^2$  is efficiently PAC-learnable.

## 5.10 The General Model - Two Kinds of Errors

The overall picture of our learning algorithm  $A$  should be as follows.



Now the pipeline has two new elements, which are:

1. **Confidence**  $\delta$ , error induced by the learning algorithm when it returns the classifier  $C$ .
2. **Accuracy**  $\epsilon$ , error produced not anymore over the training data but after the usage of the testing data.

## 5.11 Representation classes

In our definition of **efficient PAC learning**, the algorithm  $A$  has no access to the target concept  $c \in C$ , what we really want to learn. It must work in **polynomial time independently** from  $c$ . This is unrealistic in many cases: it does not make any sense to require a polynomial time with a target class  $c$  that might be too huge.

In the standard definition of PAC learnability, we only care about the number of samples  $m$  needed to reach a certain error  $\epsilon$  with a certain probability  $1 - \delta$ . However, for **efficient PAC learnability**, we shift the focus to the **running time** of the learning algorithm.

If we consider only  $\epsilon$  and  $\delta$ , we ignore a crucial factor: the complexity of the target concept  $c$  itself.

### Example

e.g. Learning a conjunction of three boolean variables is much easier than learning a conjunction of one million of variables.

If we define the efficiency on  $\epsilon$  and  $\delta$ , we might have an algorithm that is polynomial in those values but **exponential** in the number of variables or the size of the target concept.

Without knowing the size of what we are trying to learn, we cannot truly say if an algorithm is efficient.

Therefore, we remedy as follows:

- We assume that elements of the instance space can be represented by binary strings.
- We assume concepts in  $C$  (*concept class*) can be represented by way of binary strings, and each concept  $e \in C$  requires  $\text{size}(e)$  of bits.

Following this new idea, both the instance space and the concept class need new definitions:

- $C$  as  $\bigcup_n C_n$
- $X$  as  $\bigcup_n X_n$
- Any concept in  $C_n$  takes in input elements of  $X_n$

The pair  $(X, C)$  is called a **representation class**, introduced to formalize the size of the target concept,  $\text{size}(c)$ .

### Example

e.g.  $X_n$  could be  $\{0, 1\}^n$ , the set of **boolean vectors** of fixed length  $n$ , and  $C_n$  is the set of all subsets of  $\{0, 1\}^n$  *represented by CNFs* on precisely  $n$  variables.

In many cases, we have a single learning algorithm that works for every value of  $n$ . In that case, we can state that a learning task is feasible in **polynomial time** if the algorithm's running time is a polynomial function of all the following parameters:

- $n \rightarrow$  size of the input space
- $\text{size}(c) \rightarrow$  size of the target concept
- $\frac{1}{\epsilon} \rightarrow$  inverse of the error threshold (*accuracy*)
- $\frac{1}{\delta} \rightarrow$  inverse of the failure probability (*confidence*)

Note: briefly in our definition of **PAC learnability** we need to quantify also the target concept  $c$ ,  $\text{size}(c)$ . This is possible treating every concept that belongs to the concept class  $C$  as a binary string: binary strings allow us to consider the learning task like a standard **Turing Machine** problem.

## 5.12 Boolean Functions as a Representation Class

### Example

e.g. Suppose our instance class is  $X = \bigcup_n X_n$  where  $X_n = \{0, 1\}^n$ .

1<sup>st</sup> Example.

One first example of a representation class for  $X_n$  is the class  $CL_n$  of all conjunctions of literals on the variables  $x_1, \dots, x_n$ .

As an example, the conjunction

$$x_1 \wedge \neg x_2 \wedge x_4$$

defines a subset of  $\{0, 1\}^4$ .

2<sup>nd</sup> Example.

A second example of a representation class for  $X_n$  is the class  $\text{CNF}_n$  of CNFs over  $x_1, \dots, x_n$ , which are **conjunction** of **disjunctions** of literals.

## 5.13 Learning Conjunction of Literals

Suppose that our target concept is a conjunction of literals  $c$  on  $n$  variables  $x_1, \dots, x_n$ . How could a learning algorithm proceed? Or better, is this conjunction PAC learnable?

Data are in the form  $(s, b)$  where  $s \in \{0, 1\}^n$  and  $b \in \{0, 1\}$ . The latter is a label telling us whether  $s \in c$  or  $s \notin c$ , so if the binary string  $s$  is correct or not. A learning algorithm could proceed by keeping a conjunction of literals  $h$  as its state, where  $h$  is a **hypothesis**, initially set to:

$$x_1 \wedge \neg x_1 \wedge x_2 \wedge \neg x_2 \wedge \dots \wedge x_n \wedge \neg x_n$$

Then, it will update this conjunction of literals according to the positive data received, while negative data are discarded. For instance, if  $n = 3$ , so we are just keeping a conjunction of three literals, the hypothesis  $h$  could be  $x_1 \wedge x_2 \wedge \neg x_2 \wedge \neg x_3$ . If we receive a pair  $(101, 1)$ , the hypothesis  $h$  is updated as  $x_1 \wedge \neg x_2$ .

### Theorem

The representation class of boolean conjunctions of literals is efficiently PAC-learnable.

## 5.14 Intractability of Learning DNFs

We know that conjunctions of literals are **efficiently learnable**. However, they are highly incomplete as a way to represent boolean functions. As we already said in previous section, the satisfiability of conjunctions of literals is a polynomial task, but learning them has the same complexity?

Let us take a look at a slight generalization of conjunctions of literals as a representation class. A 3-term DNF formula over  $n$  bits is a propositional formula in the form  $T_1 \vee T_2 \vee T_3$ , where each  $T_i$  is a conjunction of literals over  $x_1, \dots, x_n$  boolean variables. In a sense, this class or representation class is the **dual** of 3CNFs.

For sure, this new slight generalization is more expressive than just a conjunction of literals, but it doesn't exist an algorithm that can learn the previous expression in polynomial time.



### Theorem

If  $\text{NP} \neq \text{RP}$ , then the representation class of 3-term DNF formulas is not efficiently PAC learnable.

# Chapter 6

## Occam's Razor and Consistent Learning

### 6.1 Consistent Learning Algorithms

A learning algorithm  $A$  is **consistent** if the following definition is satisfied.

#### Definition

For every training set  $D$ , it outputs a hypothesis  $h$  such that  $h(x) = c(x)$  for all pairs  $(x, c(x)) \in D$ .

In other words, by the previous definition we are just stating that  $A$  produces a hypothesis  $h$  with **zero training error**. We don't really like such property since it seems to describe a definition about **overfitting**: our learning algorithm  $A$  is so good that it provides a hypothesis  $h$  that is equal to the target concept.

However, there may be many consistent hypotheses: which one should we choose?

### 6.2 Occam's Razor in Learning

**Occam's Razor** is the principle that, among competing explanations, the **simplest one** should be preferred.

In machine learning, this refers to:

1. Take always **simpler hypothesis** that fit the data.
2. Avoid unnecessarily complex classifiers.

Specifically, it states that any hypothesis that is consistent with a sufficiently larger set of training data is very likely to generalize well to unseen data, provided it comes from a relatively small hypothesis space. The main intuition can be divided into:

1. A hypothesis that is **too complex** may **overfit** the data, such classifier provides good results only on training data.
2. Simpler hypotheses are more likely to **generalize**, performing well also on unseen data.

## 6.3 Occam's Razor for Consistent Learners

Assume that  $A$  always outputs a hypothesis consistent with  $D$ . Occam's Razor suggests:

among all consistent hypotheses, choose the **simplest**.

**Simplicity** is formalized mathematically in two distinct ways:

1. **Description length**, how many bits it takes to write it down using our representation language.
2. **Size** of the hypothesis, number of hyper-parameters required.

The take-away message must be:

fewer simple hypotheses  $\rightarrow$  less chance of fitting noise, so **overfitting**.

Simple hypotheses guarantee **better generalization behaviors**.

## 6.4 Why Simplicity Helps Generalization

Formalizing what we have seen so far, we can say.

### Definition

Let  $H$  be a finite **hypothesis class** or **space** and we consider a consistent hypothesis  $h \in H$ . If  $h$  has high true error, namely the probability it matches all  $m$  samples is small, and considering an union bound over  $H$ , where large hypothesis classes increase the risk of **overfitting**, we get:

- Smaller  $H \rightarrow$  guarantees better generalization.
- Sample complexity depends on  $\log|H|$ .

## 6.5 Occam's Razor and PAC Learning

### Theorem (Occam's Razor, Cardinality Version)

Let  $C$  be a concept class and  $H$  a hypothesis class. Let  $L$  be a consistent learner for  $C$  using  $H$ . Then for all  $0 < \epsilon < \frac{1}{2}$  and all  $0 < \delta < \frac{1}{2}$ , if  $L$  is given a sample of size  $m$  drawn from  $EX(c, \mathbf{D})$ , such that

$$m \geq \frac{1}{\epsilon} (\log |H_n| + \log \frac{1}{\delta}),$$

then  $L$  is guaranteed to output a hypothesis  $h \in H_n$  that with probability at least  $1 - \delta$ , errs with probability at most  $\epsilon$ . If furthermore,  $L$  is an efficient consistent learner,  $\log |H_n|$  is polynomial in  $n$  and  $\text{size}(c)$ , and  $H$  is polynomially evaluatable, then  $C$  is efficiently PAC-learnable using  $H$ .

This new theorem (*Occam's Razor, Cardinality Version*) has all what we have seen so far about **PAC learnability** with some new few things, which are:

1. First of all, we state that the consistent learner  $L$  learns properties from concept class  $C$  returning elements of the hypothesis class  $H$ . In this case,  $C$  and  $H$  are two different things:  $L$  learns  $C$  by  $H$ .
2. In our definition of the lower bound ( $m \geq \frac{1}{\epsilon} (\log |H_n| + \log \frac{1}{\delta})$ ) we have to take in account the cardinality of the size of the hypothesis class  $H_n$ : bigger hypothesis space would require a bigger lower bound and at the same time more training data.
3. The theorem tells us, up to the second part, that it's fine to came up with a consistent learner  $L$ , but it would expect a lot of training data if it comes from a huge hypothesis class  $H$ .
4. Additionally, if the consistent learner  $L$  runs in polynomial time, the cardinality or the size of the hypothesis class  $\log |H_n|$  is polynomial in size of  $(n, c)$ , and also the hypothesis space  $H$  is polynomially evaluatable (*meaning that feeding it with data  $x$  we will get a result in a reasonable amount of time*) then the concept class  $C$  would be efficiently PAC-learnable using the hypothesis space  $H$ .

Summing up, what the theorem is trying to tell us is that exploit **consistent learners** can be a good idea but everything depends from the cardinality of the hypothesis class  $H_n$ : if its size is big then the amount of training data we need to reach a certain level of accuracy and confidence could be even bigger.

# Chapter 7

## VC dimension

**VC dimension** is a more expressive tool than what we have seen so far. It allows us to design the **learning algorithm** and study the **concept class** by a mathematical point of view.

### 7.1 VC-Dimension: Definition

#### Definition

Let  $H$  be a hypothesis class over an instance space  $X$ . A set  $S = \{x_1, \dots, x_m\} \subseteq X$  is said to be **shattered** by the hypothesis class  $H$  if:

- For every labeling  $l : S \rightarrow \{0, 1\}$ .
- There exists  $h \in H$  such that  $h(x) = l(x)$  for all  $x \in S$ .

Basically, if the hypothesis class is sufficiently expressive to catch all the subsets of  $S$  we can state that the latter is shattered by  $H$ .

The **VC-dimension** of  $H$  is:

$$VC(H) = \max\{|S| : S \text{ is shattered by } H\}$$

In order to claim that our hypothesis class  $H$  has a finite VC-dimension we have to:

1. First of all, we show for that number there exists an  $S$  which is shattered by  $H$ .
2. We keep going increasing the finite number, until we are not able anymore to shatter the improved set  $S$  by  $H$ .

## 7.2 Examples

### Threshold Functions on $\mathbb{R}$

Let  $H$  be the class of threshold functions on  $R$ :

$$h_a(x) = \begin{cases} 1 & \text{if } x \geq a \\ 0 & \text{otherwise} \end{cases}$$

Let's see what happens for progressively bigger sets  $S_1, S_2, \dots, S_n$  whether they can be shattered by  $H$ :

$1^{st} S_1 = \{x\}$ .

We could use two threshold functions to shatter such set:

- $h_a$  located before  $x$ .
- $h_b$  located after  $x$ .

$2^{nd} S_2 = \{x, y\}$ .

We cannot shatter such set because  $\{x\}$  is a subset of  $S_2$  and no threshold function  $h$  can shatter  $h(x) = 1$  and  $h(y) = 0$ .

More precisely, in order to obtain a VC-dimension equal to 2, we would need the following combinations  $\{00, 01, 10, 11\}$ : however, the third one is not possible because we cannot place  $x$  before the threshold and  $y$  after it, since we have  $x < y$ .

#### Conclusion.

Therefore, the VC-dimension of the class of threshold functions  $H$  is 1,  $VC(H) = 1$ .

### Intervals on $\mathbb{R}$

Let  $H$  be the class of intervals on  $R$ :

$$h_{a,b}(x) = 1 \Leftrightarrow x \in [a, b]$$

So, we return the label 1 when the point is inside the interval  $[a, b]$  and 0 otherwise.

Let's see what happens for progressively bigger sets  $S_1, S_2, \dots, S_n$  whether they can be shattered by  $H$ :

$1^{st} S_1 = \{x\}$ .

We can easily shatter such set by placing the interval  $[a, b]$  around  $x$ .

$2^{st} S_2 = \{x, y\}$ .

This set is even simpler than the previous one, since we can place the interval  $[a, b]$  in many different ways to shatter  $S_2$ :

- Not around  $x$  and  $y$ .
- Around  $x$  but not  $y$ .
- Around  $y$  but not  $x$ .
- Around both  $x$  and  $y$ .

$$3^{rd}S_3 = \{x, y, z\}.$$

$S_3$ , unlike  $S_1$  and  $S_2$ , cannot be shattered by any interval  $[a, b]$ . Let's see why:

- Place  $[a, b]$  around  $x$  but not  $y$  and  $z$ .
- Place  $[a, b]$  around  $y$  but not  $x$  and  $z$ .
- Place  $[a, b]$  around  $z$  but not  $x$  and  $y$ .
- Place  $[a, b]$  around none of the three points.
- Place  $[a, b]$  around all three points  $x, y$  and  $z$ .

But, how can we place the same interval  $[a, b]$  around  $x$  and  $z$  but not  $y$ ?

Simply, we cannot since it does not exist any interval  $[a, b]$  that contains  $x$  and  $z$  but not  $y$ , considering that  $x < y < z$ .

### Conclusion.

Therefore, the VC-dimension of the class of intervals on  $R$  is 2,  $VC(H) = 2$ .

## Axis-Aligned Rectangles in $\mathbb{R}^2$

Let  $H$  be the class of axis-aligned rectangles in  $R^2$ .

Let's see what happens for progressively bigger sets  $S_1, S_2, \dots, S_n$  whether they can be shattered by  $H$ :

$$1^{st}S_1 = \{(x, y)\}.$$

We can easily shatter such set by placing the rectangle around the point  $(x, y)$ .

$$2^{nd}S_2 = \{(x_1, y_1), (x_2, y_2)\}.$$

This set is a little bit more challenging than the previous one. Like the threshold function, we can place a rectangle in different ways:

- Around both points.
- Around none of the two points.
- Around  $(x_1, y_1)$  but not  $(x_2, y_2)$ .
- Around  $(x_2, y_2)$  but not  $(x_1, y_1)$ .

$$3^{rd}S_3 = \{(x_1, y_1), (x_2, y_2), (x_3, y_3)\}.$$

We can shatter such set in the same way we did before, by placing the rectangle around the points in different ways:

- Around all three points.
- Around none of the three points.
- Around  $(x_1, y_1)$  but not  $(x_2, y_2)$  and  $(x_3, y_3)$ .
- Around  $(x_2, y_2)$  but not  $(x_1, y_1)$  and  $(x_3, y_3)$ .
- Around  $(x_3, y_3)$  but not  $(x_1, y_1)$  and  $(x_2, y_2)$ .
- Around  $(x_1, y_1)$  and  $(x_2, y_2)$  but not  $(x_3, y_3)$ .
- Around  $(x_1, y_1)$  and  $(x_3, y_3)$  but not  $(x_2, y_2)$ .
- Around  $(x_2, y_2)$  and  $(x_3, y_3)$  but not  $(x_1, y_1)$ .

$4^{th}S_4 = \{(x_1, y_1), (x_2, y_2), (x_3, y_3), (x_4, y_4)\}$ .

Taking the four points in a complete irregular position, we can shatter  $S_4$  in the following way:

- Around all four points.
- Around none of the four points.
- Around  $(x_1, y_1)$  but not  $(x_2, y_2)$ ,  $(x_3, y_3)$  and  $(x_4, y_4)$ .
- Around  $(x_2, y_2)$  but not  $(x_1, y_1)$ ,  $(x_3, y_3)$  and  $(x_4, y_4)$ .
- Around  $(x_3, y_3)$  but not  $(x_1, y_1)$ ,  $(x_2, y_2)$  and  $(x_4, y_4)$ .
- Around  $(x_4, y_4)$  but not  $(x_1, y_1)$ ,  $(x_2, y_2)$  and  $(x_3, y_3)$ .
- Around  $(x_1, y_1)$  and  $(x_2, y_2)$  but not  $(x_3, y_3)$  and  $(x_4, y_4)$ .
- Around  $(x_1, y_1)$  and  $(x_3, y_3)$  but not  $(x_2, y_2)$  and  $(x_4, y_4)$ .
- Around  $(x_1, y_1)$  and  $(x_4, y_4)$  but not  $(x_2, y_2)$  and  $(x_3, y_3)$ .
- Around  $(x_2, y_2)$  and  $(x_3, y_3)$  but not  $(x_1, y_1)$  and  $(x_4, y_4)$ .
- Around  $(x_2, y_2)$  and  $(x_4, y_4)$  but not  $(x_1, y_1)$  and  $(x_3, y_3)$ .
- Around  $(x_3, y_3)$  and  $(x_4, y_4)$  but not  $(x_1, y_1)$  and  $(x_2, y_2)$ .

$5^{th}S_5 = \{(x_1, y_1), (x_2, y_2), (x_3, y_3), (x_4, y_4), (x_5, y_5)\}$ .

There is no subset of  $R^2$  having a cardinality of 5 that can be shattered by axis-aligned rectangles.

#### **Conclusion.**

Therefore, the VC-dimension of the class of axis-aligned rectangles in  $R^2$  is 4,  $VC(H) = 4$ .

## **7.3 Positive Result: Finite VC-Dimension Implies Learnability**

VC-dimension implies learnability, or better PAC learnability.



### Definition

Let  $H$  be a hypothesis class with a finite VC-dimension  $d$ . Then  $H$  is PAC-learnable. More precisely, there exists a constant  $c$  such that for all  $0 < \epsilon$ ,  $\delta < \frac{1}{2}$ , if

$$m \geq c \cdot \frac{1}{\epsilon} \left( d \log \frac{1}{\epsilon} + \log \frac{1}{\delta} \right),$$

then with probability at least  $1 - \delta$ , any consistent hypothesis has error  $< \epsilon$ .

## 7.4 Negative Result: Infinite VC-dimension

What happens if the VC-dimension is infinite? Whether the VC-dimension is infinite then the following theorem holds.

### Theorem

Let  $H$  be a hypothesis class with infinite VC-dimension. Then  $H$  is **not** PAC-learnable. In particular, for every algorithm  $A$  there exist distributions  $\mathbf{D}$  and target concept  $c \in H$  such that  $A$  fails to achieve arbitrarily small error with high confidence.

When a hypothesis class  $H$  has an infinite VC-dimension, it means that there is no upper limit to the number of points it can shatter.

While that sounds like a good thing for our machine learning tasks, it is actually a big vulnerability. Instead of generalizing, the algorithm simply **memorizes** the training data, fitting the **noise** rather than learning a **meaningful rule**.

An infinite VC-dimension means that the hypothesis class can fit arbitrary amount of noise, leading to overfitting, and also the generalization seen by the Occam's Razor Principle becomes impossible to achieve.